

Looking at both docs, here's what needs to be added based on all the new endpoints we built. Then I'll cover GraphQL testing tools.

Updated REST API Documentation

Base URL: <http://localhost:3001/api>

Authentication

All protected endpoints require a JWT token in the Authorization header:

```
Authorization: Bearer <token>
```

Endpoints

Authentication

POST /auth/login

Authenticate user and get JWT token.

Request Body:

```
{ "phoneNumber": "+6281234567892" }
```

Response: 200 OK

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": 3,
    "fullName": "Budi Santoso",
    "phoneNumber": "+6281234567892",
    "role": "tenant",
    "isVerifiedTenant": true,
    "unitNumber": "1703",
    "createdAt": "2026-01-21T...",
    "updatedAt": "2026-01-21T..."
  }
}
```

GET /auth/me

Get current authenticated user. **Headers:** `Authorization: Bearer <token>` **Response:** 200 OK — same user object as above.

Facilities

GET /facilities

Get all active facilities. **Response:** 200 OK

```
[
  {
    "id": 1,
    "name": "Tennis Court",
    "description": "Outdoor tennis court...",
    "pricePerHour": 50000,
    "openTime": "06:00:00",
    "closeTime": "22:00:00",
    "isActive": true,
    "createdAt": "2026-01-21T...",
    "updatedAt": "2026-01-21T..."
  }
]
```

GET /facilities/:id

Get facility by ID. **Response:** 200 OK — single facility object. **Error:** 404 Not Found

```
{ "error": "NotFoundError", "message": "Facility with id 999 not found",
  "code": "NOT_FOUND" }
```

GET /facilities/:id/slots?date=YYYY-MM-DD

Get available time slots for a facility on a specific date. **Query Parameters:** `date` (required) — format YYYY-MM-DD **Response:** 200 OK

```
{
  "facilityId": 1,
  "date": "2026-01-22T00:00:00.000Z",
  "slots": [
    { "startTime": "06:00:00", "endTime": "07:00:00", "isAvailable": false },
    { "startTime": "07:00:00", "endTime": "08:00:00", "isAvailable": true }
  ]
}
```

POST /facilities

Create a new facility. **Admin only.** Headers: `Authorization: Bearer <token>` Request Body:

```
{
  "name": "Swimming Pool",
  "description": "Olympic-size indoor pool",
  "pricePerHour": 75000,
  "openTime": "06:00:00",
  "closeTime": "21:00:00",
  "isActive": true
}
```

Response: `201 Created` — facility object.

PATCH /facilities/:id

Update an existing facility. **Admin only.** All fields are optional — only send what you want to change.

Headers: `Authorization: Bearer <token>` Request Body:

```
{
  "pricePerHour": 80000,
  "isActive": false
}
```

Response: `200 OK` — updated facility object. Error: `404 Not Found` if facility does not exist.

DELETE /facilities/:id

Permanently delete a facility. **Admin only.** Headers: `Authorization: Bearer <token>` Response: `204 No Content` Error: `404 Not Found` if facility does not exist.

Bookings

GET /bookings

Get all bookings in the system. **Admin only.** Headers: `Authorization: Bearer <token>` Response: `200 OK`

```
[
  {
    "id": 301,
    "userId": 3,
    "facilityId": 1,
    "bookingDate": "2026-01-22T00:00:00.000Z",
    "startTime": "14:00:00",
```

```
    "endTime": "15:00:00",
    "status": "pending",
    "totalPrice": 0,
    "createdAt": "2026-01-21T...",
    "updatedAt": "2026-01-21T..."
  }
]
```

POST /bookings

Create a new booking. **Headers:** `Authorization: Bearer <token>` **Request Body:**

```
{
  "userId": 3,
  "facilityId": 1,
  "bookingDate": "2026-01-22",
  "startTime": "14:00:00",
  "endTime": "15:00:00"
}
```

Response: 201 Created

```
{
  "id": 301,
  "userId": 3,
  "facilityId": 1,
  "bookingDate": "2026-01-22T00:00:00.000Z",
  "startTime": "14:00:00",
  "endTime": "15:00:00",
  "status": "pending",
  "totalPrice": 0,
  "createdAt": "2026-01-21T...",
  "updatedAt": "2026-01-21T..."
}
```

Errors: 409 on conflict or daily limit exceeded. 400 on validation failure.

GET /bookings/:id

Get booking by ID. **Headers:** `Authorization: Bearer <token>` **Response:** 200 OK — single booking object (flat, no user/facility nested).

GET /bookings/:id/details

Get booking with full nested user and facility data. **Headers:** `Authorization: Bearer <token>` **Response:** 200 OK — booking object with `user` and `facility` nested inside.

PUT /bookings/:id/confirm

Confirm a pending booking. **Headers:** `Authorization: Bearer <token>` **Response:** 200 OK — booking object with `status: "confirmed"`. **Error:** 400 if booking is not in `pending` state.

DELETE /bookings/:id

Cancel a booking (sets status to `cancelled`, does not delete the record). **Headers:** `Authorization: Bearer <token>` **Response:** 200 OK — booking object with `status: "cancelled"`. **Error:** 400 if booking is already cancelled or expired.

DELETE /bookings/:id/hard

Permanently delete a booking record from the database. **Admin only.** **Headers:** `Authorization: Bearer <token>` **Response:** 204 No Content **Error:** 404 Not Found if booking does not exist.

Users

GET /users

Get all users in the system. **Admin only.** **Headers:** `Authorization: Bearer <token>` **Response:** 200 OK

```
[
  {
    "id": 3,
    "fullName": "Budi Santoso",
    "phoneNumber": "+6281234567892",
    "role": "tenant",
    "isVerifiedTenant": true,
    "unitNumber": "1703",
    "createdAt": "2026-01-21T...",
    "updatedAt": "2026-01-21T..."
  }
]
```

GET /users/:id

Get user by ID. **Headers:** `Authorization: Bearer <token>` **Response:** 200 OK — single user object.

GET /users/:id/bookings

Get all bookings for a user, with nested facility and user data. **Headers:** `Authorization: Bearer <token>` **Response:** 200 OK — array of `BookingWithDetails`.

GET /users/:id/dashboard

Get user dashboard with booking statistics. **Headers:** `Authorization: Bearer <token>` **Response:** 200 OK

```
{
  "user": { ... },
  "bookings": [ ... ],
  "bookingCountToday": 1,
  "upcomingBookings": 3,
  "totalSpent": 0
}
```

POST /users

Create a new user. **Admin only.** Headers: `Authorization: Bearer <token>` Request Body:

```
{
  "fullName": "Siti Rahayu",
  "phoneNumber": "+6289876543210",
  "role": "tenant",
  "isVerifiedTenant": true,
  "unitNumber": "0802"
}
```

Response: 201 Created — user object. **Error:** 409 Conflict if phone number is already registered.

PATCH /users/:id

Update a user. **Admin only.** All fields optional. Headers: `Authorization: Bearer <token>` Request Body:

```
{
  "isVerifiedTenant": true,
  "unitNumber": "1203"
}
```

Response: 200 OK — updated user object. **Error:** 404 Not Found, or 409 Conflict if new phone number is already taken.

DELETE /users/:id

Permanently delete a user. **Admin only.** Headers: `Authorization: Bearer <token>` **Response:** 204 No Content **Error:** 404 Not Found.

Error Responses

```
{  
  "error": "ErrorType",  
  "message": "Human-readable error message",  
  "code": "ERROR_CODE"  
}
```

HTTP status codes: 200 success, 201 created, 204 no content, 400 validation, 401 unauthenticated, 403 forbidden, 404 not found, 409 business rule conflict, 500 server error.
